Faculty of Engineering & Technology
Department of Electrical & Computer Engineering
ENCS3390
Operating System Concepts
Second Semester, 2024/2025

Project Title: Multithreaded Sudoku Validator (9x9 and 25x25)

Prepared by:

Shaimaa Dar Taha (1222440)

Abeer Hussien (1220425)

Supervised by: Dr. Yazan Abu Farha

Section: 1

Date: 01/05/2025

Abstract:

This project utilized a multithreaded application for validating the solution of a 9x9 Sudoku puzzle in the first part and 25x25 Sudoku puzzle for the second part using the C programming language. The system checks whether each row, column, and sub grids contain all digits from 1 to the size of the Sudoku puzzle exactly once, as required by Sudoku rules. In this project we implemented and compared by Three approaches a naïve, an 11 thread approach, and a 27 thread approach. Also, for the second part we implemented three approaches a naïve, a 27 thread approach, and a 75 thread approach. The core objective is to apply thread-based validation to demonstrate concurrency and parallelism concepts as learned in the Operating System Concepts course. The project includes different levels of threading to compare performance and efficiency.

Contents

Abstract: 1

Table of figures: 3

List of tables: 3

Theory: 1

- **Implementation Overview:** 1
- **Validation Methods:** 1
- **Analysis and Comparison:** 2
  - **Key Insights:** 4

Environment Description: 4

- **Processor** 4
- **Memory** 4
- **Operating system** 4
- **Programming Language and IDE tool:** 4

Scalability Challenges: 5

Conclusion: 5

Table of figures:

Figure 1: results for valid Sudoku run the program 2
Figure 2: results for not valid Sudoku run the program 3

List of tables:

Table 1: result for valid Sudoku run the program 2
Table 2: result for not valid Sudoku run the program 3

Theory:

Implementation Overview:

Before diving into implementation details, it is important to emphasize that the project was designed with scalability, performance, and user flexibility in mind. The program allows users to select between two grid sizes (9x9 and 25x25) and choose among multiple validation approaches tailored to each. This modular design ensures clarity in execution and allows for fair performance comparisons between single-threaded and multithreaded models.

The implementation is modular and user-interactive. The user can choose between validating a 9x9 or 25x25 grid and then select one of three validation methods. The grid is loaded from a text file and analyzed using multiple threads as per the selected method.

Validation Methods:

- **Naive Approach (Single Thread):**

· Checks all rows, columns, and subgrids in a sequential manner.

- **Moderate Multithreading:**

· **9x9: 11 threads** (1 thread for all rows, 1 for all columns, 9 for individual subgrids).

· **25x25: 27 threads** (1 for all rows, 1 for all columns, 25 for individual subgrids).

- **Full Multithreading:**

· **9x9: 27 threads** (9 thread for all rows, 9 for all columns, 9 for individual subgrids).

· **25x25: 75 threads** (25 thread for all rows, 25 for all columns, 25 for individual subgrids).

- **The execution time** is calculated using the clock () function in C by include <time.h> library to provide a clear comparison across the methods.

Analysis and Comparison:

One valid Sudoku run example:

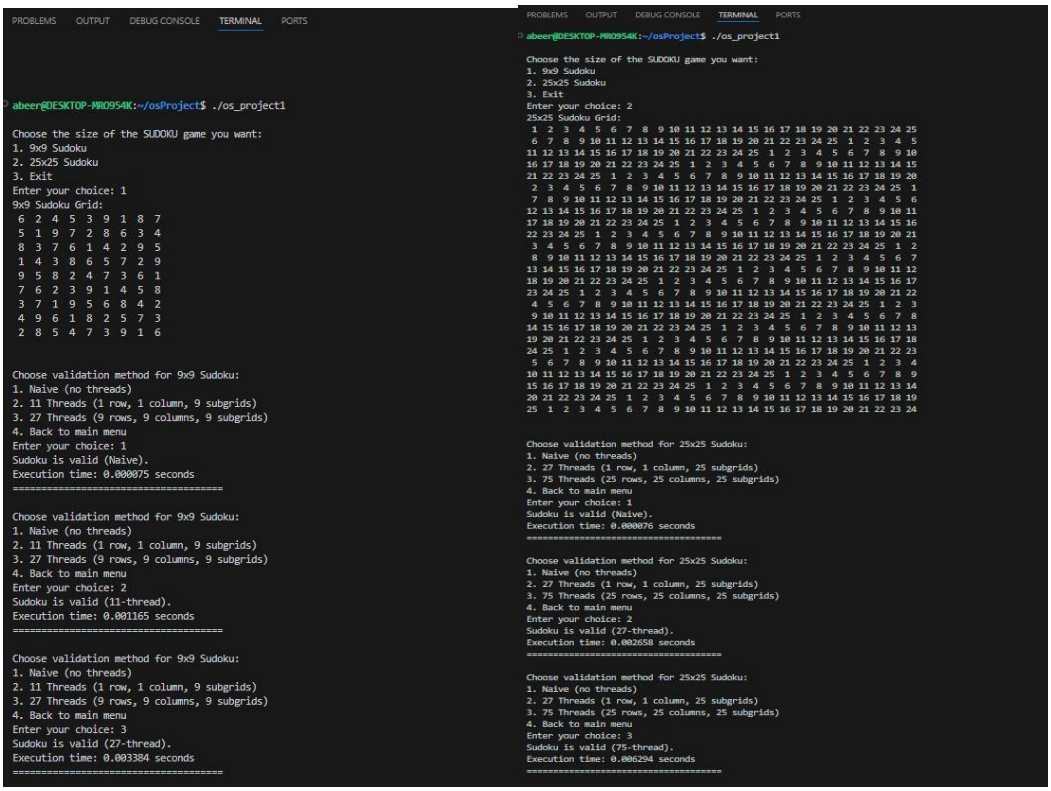
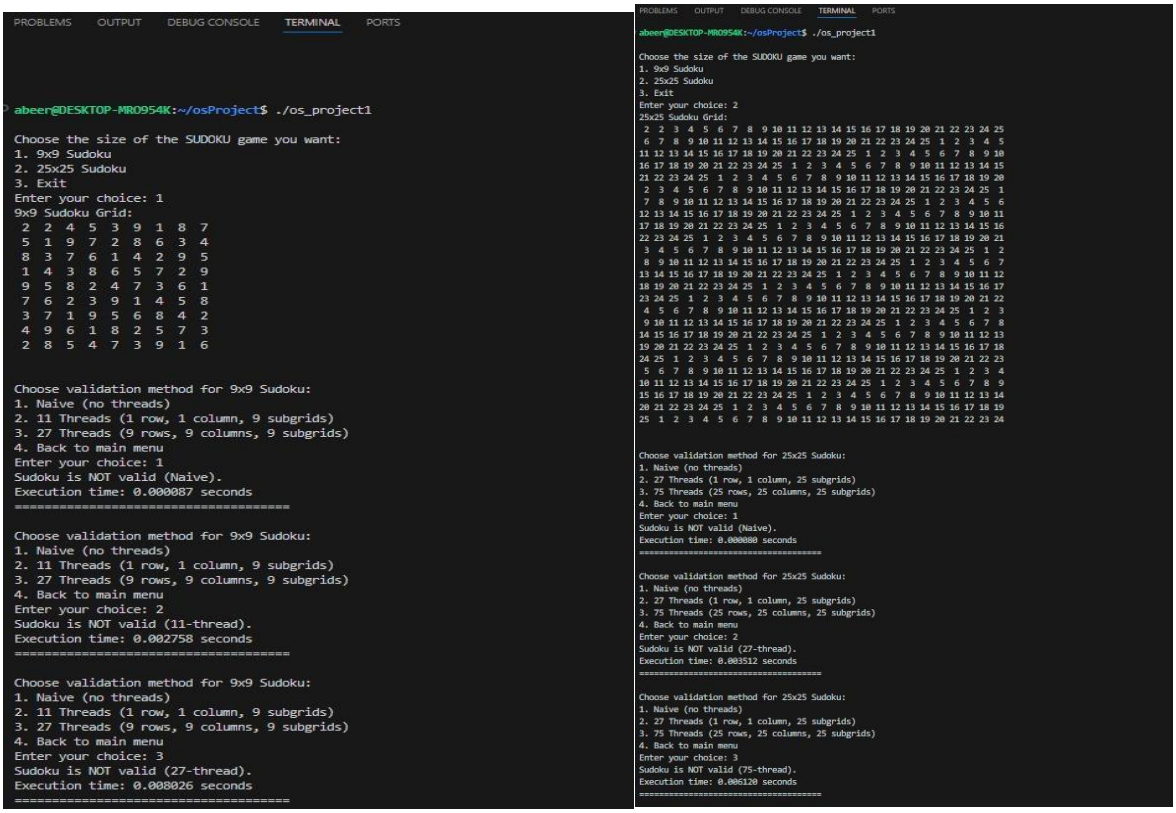


Figure 1: results for valid Sudoku run the program

Grid Size	Method	Threads	Time (s)	Result
9x9	Naive	0	0.000075	Valid
9x9	11 threads	11	0.001165	Valid
9x9	27 threads	27	0.003384	Valid
25x25	Naive	0	0.000076	Valid
25x25	27 threads	27	0.002658	Valid
25x25	75 threads	75	0.006264	Valid

Table 1: result for valid Sudoku run the program

One not valid Sudoku run example:



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

abeer@DESKTOP-MRO954K:~/osProject$ ./os_project1

Choose the size of the SUDOKU game you want:
1. 9x9 Sudoku
2. 25x25 Sudoku
3. Exit
Enter your choice: 1
9x9 Sudoku Grid:
 2  2  4  5  3  9  1  8  7
 5  1  9  7  2  8  6  3  4
 8  3  7  6  1  4  2  9  5
 1  4  3  8  6  5  7  2  9
 9  5  8  2  4  7  3  6  1
 7  6  2  3  9  1  4  5  8
 3  7  1  9  5  6  8  4  2
 4  9  6  1  8  2  5  7  3
 2  8  5  4  7  3  9  1  6

Choose validation method for 9x9 Sudoku:
1. Naive (no threads)
2. 11 Threads (1 row, 1 column, 9 subgrids)
3. 27 Threads (9 rows, 9 columns, 9 subgrids)
4. Back to main menu
Enter your choice: 1
Sudoku is NOT valid (Naive).
Execution time: 0.000087 seconds
=====================================

Choose validation method for 9x9 Sudoku:
1. Naive (no threads)
2. 11 Threads (1 row, 1 column, 9 subgrids)
3. 27 Threads (9 rows, 9 columns, 9 subgrids)
4. Back to main menu
Enter your choice: 2
Sudoku is NOT valid (11-thread).
Execution time: 0.002758 seconds
=====================================

Choose validation method for 9x9 Sudoku:
1. Naive (no threads)
2. 11 Threads (1 row, 1 column, 9 subgrids)
3. 27 Threads (9 rows, 9 columns, 9 subgrids)
4. Back to main menu
Enter your choice: 3
Sudoku is NOT valid (27-thread).
Execution time: 0.008026 seconds
=====================================
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

abeer@DESKTOP-MRO954K:~/osProject$ ./os_project1

Choose the size of the SUDOKU game you want:
1. 9x9 Sudoku
2. 25x25 Sudoku
3. Exit
Enter your choice: 2
25x25 Sudoku Grid:
 2  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
 6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5
11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10
16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
 2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1
 7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6
12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11
17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21
 3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2
 8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7
13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12
18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22
 4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3
 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8
14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13
19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4
10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9
15 16 17 18 19 20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14
20 21 22 23 24 25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
25  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24


Choose validation method for 25x25 Sudoku:
1. Naive (no threads)
2. 27 Threads (1 row, 1 column, 25 subgrids)
3. 75 Threads (25 rows, 25 columns, 25 subgrids)
4. Back to main menu
Enter your choice: 1
Sudoku is NOT valid (Naive).
Execution time: 0.000080 seconds
=====================================

Choose validation method for 25x25 Sudoku:
1. Naive (no threads)
2. 27 Threads (1 row, 1 column, 25 subgrids)
3. 75 Threads (25 rows, 25 columns, 25 subgrids)
4. Back to main menu
Enter your choice: 2
Sudoku is NOT valid (27-thread).
Execution time: 0.003512 seconds
=====================================

Choose validation method for 25x25 Sudoku:
1. Naive (no threads)
2. 27 Threads (1 row, 1 column, 25 subgrids)
3. 75 Threads (25 rows, 25 columns, 25 subgrids)
4. Back to main menu
Enter your choice: 3
Sudoku is NOT valid (75-thread).
Execution time: 0.006120 seconds
=====================================
```

Figure 2: results for not valid Sudoku run the program

Grid Size	Method	Threads	Time (s)	Result
9x9	Naive	0	0.000087	Not Valid
9x9	11 threads	11	0.002758	Not Valid
9x9	27 threads	27	0.008026	Not Valid
25x25	Naive	0	0.000080	Not Valid
25x25	27 threads	27	0.003512	Not Valid
25x25	75 threads	75	0.006120	Not Valid

Table 2: result for not valid Sudoku run the program

Key Insights:

- **Note on Timing Variability:** It's important to recognize that execution time may vary slightly with each run due to external system factors such as CPU load, background processes, thread scheduling, and memory cache behavior. These factors introduce minor fluctuations even if the input and logic remain unchanged.
- **The naive approach** is simple and effective for small grids but becomes inefficient for larger ones.
- **Moderate threading** balances performance and complexity well.
- **Full threading**, while offering parallelism, does **not always result in better performance** due to thread overhead and context switching. This explains why more threads sometimes lead to **longer execution times**.

Environment Description:

Processor

The device we worked on is equipped with an 11th Generation Intel Core i5-1135G7 processor. This CPU has 4 cores and 8 logical cores (threads). The processor operates at a base frequency of 2.42 GHz, with dynamic speed adjustments depending on system workload. It includes 13.3 MB. The system supports 64-bit architecture.

Memory

The system is equipped with 16 GB of DDR4 RAM, running at a frequency of 3200 MT/s. the system can achieve a theoretical maximum memory bandwidth of approximately 51.2 GB/s.

Operating system

The Operating System of the device is Windows 10, 64-bit based.

Programming Language and IDE tool:

The implementation of this project was carried out using the C programming language due to its performance and fine-grained control over memory and threading. The code was compiled using the GCC (GNU Compiler Collection) and executed within a Linux environment using the Windows Subsystem for Linux (WSL) integrated in Visual Studio Code. Multithreading was implemented using POSIX threads (Pthreads), allowing parallel validation to improve performance.

Scalability Challenges:

Scaling from 9x9 to 25x25 involved several design and performance challenges:

- **Memory Consumption:** The 25x25 grid requires significantly more memory, especially when using large arrays and result buffers.
- **Thread Management:** Creating and joining 75 threads introduces noticeable overhead. Threads must be carefully synchronized and properly freed.
- **CPU Utilization:** On machines with limited logical cores, excessive threads may lead to context switching, impacting performance.
- **Input Formatting:** The 25x25 input files are large and need strict formatting to avoid parsing errors during validation.

Conclusion:

This project demonstrates how multithreading can enhance computational performance in validating Sudoku puzzles. Through the use of POSIX threads in C, we implemented three methods to verify Sudoku correctness. Our tests confirmed that while single-threaded solutions are acceptable for smaller grids, multithreading provides performance improvements up to a certain point.

However, we also observed that more threads do not always mean better performance. When thread count exceeds what the hardware can efficiently handle, the execution time may increase due to thread overhead. Therefore, it is crucial to balance concurrency with hardware limitations.

The structured approach also helped reinforce core concepts in concurrency, thread safety, and scalability. Each method has trade-offs in simplicity, performance, and resource usage, making the choice of approach dependent on the application requirements.